

Team 1 Project Documentation

USC Study Room Booking Platform

I. Project Proposal

Team Members & Emails:

1. Yucheng Zhao - yzhao635@usc.edu
2. Rakshay Jain - rakshayj@usc.edu
3. William Wang - wwang767@usc.edu
4. Seungho Lee - seunghol@usc.edu
5. Rohan Ramakrishnan - rohanram@usc.edu
6. Caroline Lo - locaroli@usc.edu
7. Isabella Crasto - crasto@usc.edu
8. AJ Chaikovsky - achaikov@usc.edu

Weekly Meeting Time: Mon 8:30PM

Project Proposal: (1-2 sentences)

Platform for USC students to browse and book study rooms across campus with real-time availability and reviews. Possible additional features include adding ratings, study group matching, and map of available locations.

II. High Level Requirements

Introduction

This document outlines the high-level requirements for a new software application designed to help students book campus study rooms. These requirements originate from the perspective of the students requesting the software and purposefully leave out details of the technical implementation.

User Access and Authentication

The system needs to support two distinct experiences based on user status. Guests will have limited access to the platform and can only view the available study rooms and the campus map. Authenticated users can book rooms and leave reviews, while also interacting with the social features of the application, after logging into the system. Users can create accounts and log in to them to access the authenticated user experience.

User Profiles and Social Matching

Registered users will have profiles that display their current courses and social preferences (edited by the user). The platform requires a matching system to connect students who want to meet new people or form study groups. This social layer will help students find classmates taking the same courses and encourage collaborative learning.

Study Room Availability and Discovery

The core feature of the application is a real time availability page. Users need to see a list of all open study rooms across campus. This list should sort by building and room number by default but users must also be able to sort the list by room ratings. The platform requires a map view showing exactly where these rooms are located. An optional heatmap could be included to show which campus areas are currently the busiest. Users also need the ability to filter the available rooms based on specific resources like whiteboards or monitors.

Booking System and Restrictions

Authenticated users need the ability to reserve available rooms in real time. The booking interface must show all the features of a room before the student confirms the reservation. Users must be able to cancel their reservations easily if their plans change.

The platform needs specific rules to prevent people from hoarding rooms. A single user cannot reserve rooms in the same building for more than two hours. Users are also strictly limited to one reservation per time slot. These specific booking limits apply to individual users and do not apply to entire study groups.

Waitlists and Reviews

A waitlist system is required for popular rooms that are already booked. If a student cancels a reservation the system must automatically move through the waitlist and give the room to the next person in line. The platform also needs a rating system. Authenticated users must be able to leave reviews for rooms they have used so other students know what to expect regarding noise levels and cleanliness.

III. Technical Specifications:

Web Interface (7 hours)

Login Interface

- “Continue as Guest” button, user will continue to Unauthorized User Interface
- Username field
- Password field
- Toggle between Login and Account Creation
- Login/Create Account button
- After user enters these fields, reference database to validate user and switch to the Authorized User Interface if successful

Unauthorized User Interface

- Drop-down list of available study rooms
- Campus map display
- Login button

Authorized User Interface

- Drop-down list of available study rooms
 - Campus map display
 - Book a room
- Logout button
- Manage my bookings
- Display option to leave reviews
 - Write a new review
 - Edit my reviews
- Display option for social features

Social Matching Interface

- List of recommended study partners based on overlapping customers stored in user profiles
- Show the shared courses between the current user and the recommended match
- Search bar allowing users to search for other students by course name
- Button allowing users to send a study request to another student
- List of incoming study requests received by the user, each including options to accept or decline

Waitlist/Booking System (8 hours)

- Heatmap of rooms based on occupancy
- Filtering system based on features of the room
- Display of number of students in the waitlist
- Field to add name to waitlist
- Cancellation button
- Rating displayed
 - Subcategories: noise, cleanliness, etc

- Sorting dropdown
- View reviews list for each room

Database (10 hours)

- User Table
 - userID
 - userName
 - passwordHash
 - firstName
 - lastName
 - role
 - courses
 - socialPreferences
 - lastLoginTimestamp
- Room Table
 - roomID
 - buildingName
 - roomNumber
 - capacity
 - featureList
 - mapLocation
 - currentStatus
 - averageRating
 - ratingsCount
- Reservation Table
 - reservationID
 - userID
 - roomID
 - startTime
 - endTime
 - status
 - createdTimestamp
- Waitlist Table
 - waitlistID
 - userID
 - roomID
 - requestedTimeSlot
 - queuePosition
 - createdTimestamp
- Review Table
 - reviewID
 - userID
 - roomID
 - rating

- comment
 - createdTimestamp
- Match Table
 - matchID
 - userID
 - matchedUserID
 - sharedCourses
 - compatibilityScore
 - createdTimestamp

Hardware Requirements

- Laptop with 16 GB RAM
- Internet Connection (online deployment)

Software Requirements

- Database Management System
- Web Browser
- Web Server Environment

IV. Detailed Design

1. Overview

This document provides a comprehensive design of the USC Study Room Booking Platform. It includes system architecture, GUI design, database schema, class inheritance hierarchy, and software / hardware requirements. The goal of this platform is to allow students to efficiently book study rooms, manage reservations, and connect with study partners.

2. System Features & Key Algorithms

Core Functionalities

- User authentication (login / guest access / create account)
- Study room browsing and booking
- Waitlist management system
- Review and rating system
- Social matching system

Difficult / Key Algorithms

1. Room Availability Check

- Ensures no overlapping reservations
- Logic:
 - If ($\text{new_start} < \text{existing_end}$) AND ($\text{new_end} > \text{existing_start}$) $\rightarrow$ conflict (cannot book)

2. Waitlist Queue Management

- FIFO-based priority queue
- Automatically promotes users when a slot opens
- Updates:
 - queuePosition recalculated dynamically
 - notifications triggered (future improvement)

3. Social Matching Algorithm

- Based on:
 - Shared courses
- Example scoring:

$\text{compatibilityScore} = (\# \text{ shared courses} * \text{weight})$

4. Heatmap Generation

- Based on:
 - Current reservations
- Algorithm:
 - Define X as number of reservations of the room with the most reservations
 - Then, divide reservation count for each room by X
 - This will give 0.0-1.0 scale for each room
 - Green → closer to 0.0 (low capacity)
 - Red → closer 1.0 (high capacity)

5. Rating Calculations

- Schedule a script to recalculate and store the average ratings and total number of ratings within the Room table.
 - Average rating = total rating sum / number of ratings

3. GUI Design



a. Login Interface

- Toggle between Login and Create Account
- Username + Password fields
- Login/Create Account button
- "Continue as Guest" option

b. Unauthorized (Guest) Interface

- View available study rooms (read-only)
- Campus map visualization
- Login button

c. Authorized User Interface

- View available rooms
- Campus map
- Book a room
- Manage bookings
- Leave / edit reviews
- Access social features

d. Social Matching Interface

- Recommended study partners list
- Display shared courses
- Search bar (by course)
- Send study request button
- Incoming requests (Accept / Decline)

e. Waitlist & Booking Interface

- Room heatmap visualization
- Filters:
 - Capacity
 - Features
- Waitlist size display
- Join waitlist option
- Review display:
 - Rating
 - Subcategories (noise, cleanliness, etc.)

4. Database Schema

User Table

- userID (PK)
- userName
- passwordHash

- firstName
- lastName
- role
- courses
- socialPreferences
- lastLoginTimestamp

Room Table

- roomID (PK)
- buildingName
- roomNumber
- capacity
- featureList
- mapLocation
- currentStatus
- averageRating
- ratingsCount

Reservation Table

- reservationID (PK)
- userID (FK)
- roomID (FK)
- startTime
- endTime
- status
- createdTimestamp

Waitlist Table

- waitlistID (PK)
- userID (FK)
- roomID (FK)
- requestedTimeSlot
- queuePosition
- createdTimestamp

Review Table

- reviewID (PK)
- userID (FK)
- roomID (FK)

- rating
- comment
- createdTimestamp

Match Table

- matchID (PK)
- userID (FK)
- matchedUserID (FK)
- sharedCourses
- compatibilityScore
- createdTimestamp

5. Class Diagram & System Design

Core Classes

User

- Attributes:
 - userID, name, role, preferences
- Methods:
 - login()
 - updateProfile()

Room

- Attributes:
 - roomID, capacity, features
- Methods:
 - checkAvailability()
 - getStatus()

Reservation

- Attributes:
 - reservationID, timeSlot
- Methods:
 - createReservation()
 - cancelReservation()

Waitlist

- Attributes:
 - queuePosition
- Methods:
 - joinWaitlist()
 - promoteUser()

Review

- Attributes:
 - rating, comment
- Methods:
 - submitReview()

MatchSystem

- Methods:
 - findMatches()
 - calculateCompatibility()

Inheritance / Relationships

- User → (can have many) Reservations
- Room → (can have many) Reservations
- Room → (can have many) Reviews
- User → (can have many) Matches
- Reservation → linked to User + Room
- Waitlist → linked to User + Room

6. Hardware Requirements

- Laptop or desktop computer
- Minimum 16 GB RAM
- Stable internet connection

7. Software Requirements

- Web browser (Chrome, Safari, etc.)
- Backend server (Java Spring Boot)
- Database system (MySQL)
- Frontend framework (React)

8. Additional Notes

- System is designed to be scalable for university-wide use
- Modular architecture allows future expansion:
 - Notifications system
 - Mobile application
 - AI-based room recommendations

V. Testing Plan

1. Testing Objectives

The testing plan is designed to confirm the following:

- Guests can view available rooms and the campus map, but cannot book rooms or leave reviews.
- Authenticated users can log in, book rooms, cancel bookings, leave reviews, and use social features.
- The booking system prevents overlapping reservations and enforces the project booking restrictions.
- The waitlist correctly stores users in order and promotes the next user when a reservation is canceled.
- Reviews and ratings are stored and displayed correctly.
- The social matching system recommends users based on shared courses and allows study requests to be sent and handled.
- Sorting, filtering, and map-related room browsing features work correctly.

2. Testing Strategies

Black Box Testing

Black box testing will focus on user-visible behavior. The tester will interact with the system through the interface and verify whether the output matches the expected result.

White Box Testing

White box testing will focus on the internal logic of the system. This includes checking conditions such as overlapping reservation detection, waitlist position updates, compatibility score calculations, and permission checks for guest versus authenticated users.

Unit Testing

Unit testing will test individual methods such as `login()`, `checkAvailability()`, `createReservation()`, `cancelReservation()`, `joinWaitlist()`, `promoteUser()`, `submitReview()`, and `calculateCompatibility()`. These methods are identified in the detailed design.

Regression Testing

Regression testing will be run after any changes to the booking system, waitlist system, review system, or social matching logic. This will ensure that new code does not break previously working features.

Stress Testing

Stress testing will examine how the program handles many users, many reservations, or many waitlist entries at once. This is especially important for the booking and waitlist systems because they may be used by many students at the same time.

3. Test Cases

A. Authentication and Access Control

Test Case A1: Valid login

Type: Black box / Unit

Input: Correct username and correct password

Expected Output: User is successfully logged in and taken to the Authorized User Interface.

Test Case A2: Invalid login password

Type: Black box / Unit

Input: Correct username and incorrect password

Expected Output: Login fails and an error message is displayed.

Test Case A3: Invalid login username

Type: Black box / Unit

Input: Username that does not exist and any password

Expected Output: Login fails and an error message is displayed.

Test Case A4: Guest access

Type: Black box

Input: User clicks “Continue as Guest”

Expected Output: User is taken to the Unauthorized User Interface and can view available rooms and the map only. Booking, review, and social features are unavailable.

Test Case A5: Empty login fields

Type: Black box

Input: Blank username and/or blank password

Expected Output: Login is rejected and the user is prompted to fill in the required fields.

Test Case A6: Valid account creation

Type: Black box / Unit

Input: Username that does not exist and any password

Expected Output: Account is successfully created and page is toggled to Login.

Test Case A7: Invalid login username

Type: Black box / Unit

Input: Username that already exists and any password

Expected Output: Account creation fails and an error message is displayed.

Test Case A5: Empty login fields

Type: Black box

Input: Blank username and/or blank password

Expected Output: Account creation is rejected and the user is prompted to fill in the required fields.

B. Room Browsing and Discovery

Test Case B1: View available rooms as guest

Type: Black box

Input: Guest opens room list

Expected Output: Available rooms are displayed successfully.

Test Case B2: Sort rooms by building and room number

Type: Black box

Input: Open availability page with default sorting

Expected Output: Rooms appear sorted by building and room number.

Test Case B3: Sort rooms by rating

Type: Black box

Input: User selects sort by rating

Expected Output: Rooms are reordered from highest rated to lowest rated.

Test Case B4: Filter rooms by feature

Type: Black box

Input: User selects a feature such as whiteboard or monitor

Expected Output: Only rooms with the selected feature are displayed.

Test Case B5: Display campus map

Type: Black box

Input: User opens map view

Expected Output: Campus map is displayed and room locations are shown.

C. Room Booking

Test Case C1: Valid room reservation

Type: Black box / Unit

Input: Authenticated user selects an available room and a valid time slot

Expected Output: Reservation is created successfully and appears in the user's bookings list.

Test Case C2: Attempt overlapping reservation for same room

Type: White box / Unit

Input: New reservation where $\text{new_start} < \text{existing_end}$ and $\text{new_end} > \text{existing_start}$

Expected Output: Reservation is rejected because the time overlaps an existing reservation.

Test Case C3: One reservation per time slot rule

Type: Black box / White box

Input: User already has a reservation during a time slot and attempts to reserve another room for the same time slot

Expected Output: Reservation is rejected based on the project rule.

Test Case C4: Same building over two-hour limit

Type: Black box / White box

Input: User attempts to reserve rooms in the same building for more than two hours total

Expected Output: Reservation is rejected based on the booking restriction.

Test Case C5: Guest attempts to book room

Type: Black box

Input: Guest tries to click booking option

Expected Output: Booking is blocked and guests are prompted to log in.

D. Booking Management

Test Case D1: Cancel reservation

Type: Black box / Unit

Input: Authenticated user cancels an active reservation

Expected Output: Reservation status is updated and the room becomes available or the waitlist is processed.

Test Case D2: View current bookings

Type: Black box

Input: Authenticated user opens “Manage My Bookings”

Expected Output: User sees a list of their current reservations.

E. Waitlist System

Test Case E1: Join waitlist for full room

Type: Black box / Unit

Input: Authenticated user selects a room that is unavailable and chooses to join the waitlist

Expected Output: User is added to the waitlist with the correct queue position.

Test Case E2: Waitlist queue order

Type: White box / Unit

Input: Three users join the same waitlist in sequence

Expected Output: Queue positions are assigned in FIFO order as 1, 2, and 3.

Test Case E3: Promotion after cancellation

Type: White box / Regression

Input: First reservation is canceled while users are on the waitlist

Expected Output: The first user in the waitlist is promoted to the reservation and queue positions update correctly.

Test Case E4: Display waitlist size

Type: Black box

Input: User views a room with a waitlist

Expected Output: The system displays the correct number of students on the waitlist.

F. Reviews and Ratings**Test Case F1: Submit review as authenticated user**

Type: Black box / Unit

Input: Logged-in user enters rating and comment for a room

Expected Output: Review is saved and displayed for that room.

Test Case F2: Guest attempts to leave review

Type: Black box

Input: Guest tries to access review submission

Expected Output: Action is blocked and guest is asked to log in.

Test Case F3: Edit existing review

Type: Black box / Unit

Input: Authenticated user edits their previous review

Expected Output: Updated rating and comment are stored and displayed correctly.

Test Case F4: Average room rating calculation

Type: White box / Unit / Regression

Input: Multiple reviews are stored for one room

Expected Output: The room's average rating equals total ratings divided by number of reviews.

G. Social Matching Features**Test Case G1: Display recommended study partners**

Type: Black box

Input: Authenticated user opens social matching page

Expected Output: System displays recommended users based on overlapping courses.

Test Case G2: Show shared courses

Type: Black box

Input: User views a recommended match

Expected Output: Shared courses between the current user and the matched user are shown.

Test Case G3: Search by course name

Type: Black box

Input: User enters a course name into the search bar

Expected Output: The system returns users taking that course.

Test Case G4: Send study request

Type: Black box / Unit

Input: User clicks “Send Study Request” on another student

Expected Output: Request is stored and visible to the receiving user.

Test Case G5: Accept study request

Type: Black box / Unit

Input: Recipient clicks “Accept” on an incoming request

Expected Output: Request status updates successfully.

Test Case G6: Decline study request

Type: Black box / Unit

Input: Recipient clicks “Decline” on an incoming request

Expected Output: Request status updates successfully.

Test Case G7: Compatibility score calculation

Type: White box / Unit

Input: Two users with different numbers of shared courses

Expected Output: User pair with more shared courses receives a higher compatibility score.

H. Heatmap and Occupancy Features

Test Case H1: Heatmap display

Type: Black box

Input: User opens heatmap view

Expected Output: Rooms appear with colors representing relative occupancy or booking frequency.

Test Case H2: Heatmap scaling logic

Type: White box / Unit

Input: Rooms have different reservation counts

Expected Output: Each room is assigned a normalized value from 0.0 to 1.0 based on the room with the highest reservation count.

4. Regression Testing Plan

Regression testing will be performed after every major code update. The following features will always be retested:

- Login and guest access
- Room browsing and filtering

- Reservation creation
- Reservation cancellation
- Waitlist promotion
- Review submission
- Social matching recommendations

For example, if the booking logic is changed, the team must retest overlap detection, time slot restrictions, cancellation behavior, and waitlist promotion. If the review system is changed, the team must retest review submission, editing, and average rating calculations.

5. Stress Testing Plan

The following stress tests will be used:

Stress Test 1: Many simultaneous booking attempts

Input: Many users attempt to reserve the same room and time slot

Expected Output: Only one valid reservation is created and all other users are rejected or placed on the waitlist correctly.

Stress Test 2: Large waitlist size

Input: A large number of users join the same waitlist

Expected Output: Queue positions remain correct and the system still promotes users in the correct order.

Stress Test 3: Large number of reviews

Input: Many reviews are submitted for one room

Expected Output: Average rating remains correct and the room page still loads properly.

Stress Test 4: Large number of user matches

Input: Many users exist in the system with many overlapping courses

Expected Output: Matching results are still generated correctly and within a reasonable time.

6. Tools and Implementation Approach

The team will implement tests in code for important classes and methods, especially those related to booking, waitlists, reviews, and matching. Interface-based testing will also be completed manually to verify that the full user experience behaves correctly. Unit tests will be especially useful for methods identified in the detailed design, such as reservation creation, cancellation, waitlist promotion, and compatibility calculations.

VI. Deployment Guide

This section outlines the step-by-step procedures required to deploy and run the USC Study Room Booking Platform in a new environment. It details the necessary prerequisites, server configurations, external libraries, and execution steps for the database, backend, and frontend components.

1. Prerequisites and Environment Setup

Before beginning the deployment process, ensure the target environment meets the hardware and software requirements outlined in the design document.

Hardware Requirements

- **Computer:** Laptop, desktop, or virtual machine (VM)
- **Memory:** Minimum 16 GB RAM (required for running MySQL, Spring Boot, and React concurrently)
- **Network:** Stable internet connection for downloading dependencies and API communication

Software Requirements

Ensure the following software is installed on your system. If they are not installed, please download them from their respective official websites:

1. **Java Development Kit (JDK):** Version 11 or 17 (Required for Java Spring Boot).
 2. **Node.js & npm:** Version 14.x or higher (Required for the React frontend).
 3. **MySQL Server:** Version 8.0 or higher.
 4. **Git:** For cloning the source code repository.
 5. **Maven (Optional):** If not using the included Maven wrapper ([mvnw](#)).
-

2. Database Configuration (MySQL)

The platform relies on a relational database schema containing [User](#), [Room](#), [Reservation](#), [Waitlist](#), [Review](#), and [Match](#) tables.

1. **Start the MySQL Server** on your machine.
2. **Open your MySQL Command Line** or a GUI client (like MySQL Workbench) and log in as the root user.
3. **Create the Database:** Run the following SQL command to create the application database.
`CREATE DATABASE usc_study_rooms;`
4. **Create a Dedicated Database User (Recommended):**

```
CREATE USER 'usc_admin'@'localhost' IDENTIFIED BY 'securepassword';
```

```
GRANT ALL PRIVILEGES ON usc_study_rooms.* TO 'usc_admin'@'localhost';  
FLUSH PRIVILEGES;
```

5. The actual table creation (schema instantiation) will be handled automatically by the Spring Boot backend using Hibernate/JPA upon its first execution.
 6. Populate all tables with the provided populate_database.sql script.
-

3. Backend Deployment (Java Spring Boot)

The backend handles the core logic, including user authentication, room availability algorithms, waitlist queues, social matching, and scheduled rating calculations.

Step 3.1: Configure Server Settings

1. Open a terminal and clone the project repository:

```
git clone <repository_url>  
cd usc-study-room-platform/backend
```
2. Navigate to `src/main/resources/` and open the `application.properties` (or `application.yml`) file.
3. Update the database credentials to match the database created in Step 2:

```
spring.datasource.url=jdbc:mysql://localhost:3306/usc_study_rooms?useSSL=false&serverTimezone=UTC  
  
spring.datasource.username=usc_admin  
spring.datasource.password=securepassword  
  
# Allows Spring to automatically generate tables based on the Entity classes  
spring.jpa.hibernate.ddl-auto=update  
  
# Server port mapping  
server.port=8080
```

Step 3.2: External Libraries & Dependencies

The backend utilizes several key dependencies (managed via `pom.xml`):

- **Spring Data JPA / Hibernate:** For database operations and ORM.
- **Spring Security:** To handle user authentication (login/guest access).
- **MySQL Connector/J:** For database communication.
- **Spring Boot Starter Web:** For RESTful API routing.
- *Note on Scheduled Scripts:* Ensure the `@EnableScheduling` annotation is active in your main application class to allow the backend to run the scheduled script that recalculates average ratings.

Step 3.3: Run the Backend Server

In the backend root directory, run the application using the Maven wrapper:

For Windows:

```
mvnw.cmd spring-boot:run
```

For Mac/Linux:

```
./mvnw spring-boot:run
```

Verify that the console outputs "Started Application" without any database connection errors. The backend should now be running on <http://localhost:8080>.

4. Frontend Deployment (React)

The React frontend handles the GUI, including the campus map visualization, heatmaps, booking interfaces, and social matching dashboards.

Step 4.1: Configure Environment Variables

1. Open a new terminal window and navigate to the frontend directory:

```
cd usc-study-room-platform/frontend
```
2. Create a `.env` file in the root of the frontend directory to link the frontend to the backend API:

```
REACT_APP_API_BASE_URL=http://localhost:8080/api
```

Step 4.2: External Libraries & Dependencies

The frontend relies on specific npm packages (defined in `package.json`). Key libraries include:

- **React Router Dom:** For navigating between Authorized, Unauthorized, and Login interfaces.
- **Axios:** For executing API requests to the Spring Boot backend.
- **Chart.js / D3.js (or similar):** For rendering the Room Heatmap based on the 0.0–1.0 capacity algorithm.

Step 4.3: Install Dependencies and Run

1. Install all required Node packages:

```
npm install
```
2. Start the React development server:

```
npm start
```

The web browser should automatically open to <http://localhost:3000>. If it does not, manually enter the URL into your browser.

5. Verification and Troubleshooting

Once both servers are running, perform a system check to ensure successful deployment:

1. **Verify Database:** Open your MySQL client and verify that the `usc_study_rooms` database now contains the auto-generated tables (`User`, `Room`, `Reservation`, `Waitlist`, `Review`, `Match`).
2. **Verify Cross-Origin Resource Sharing (CORS):** If the frontend fails to fetch data (e.g., room lists are blank), ensure that the Spring Boot backend has a `@CrossOrigin(origins = "http://localhost:3000")` configuration set up on its controllers to allow local API requests.
3. **Check Scheduled Tasks:** Verify in the backend console logs that the rating calculation script is firing successfully at its scheduled interval.
4. **Port Conflicts:** If `npm start` or `mvnw` fails due to a port already in use, terminate the conflicting process or change `server.port` in the backend (or `PORT` in the frontend `.env` file).